Reinforcement Learning Enhanced LRU Cache Replacement Policy

Implementation and Evaluation Study

Motivation

The cache replacement problem aims to maximize hit rates within a constrained memory size.

Limitations of Traditional LRU

- **Vulnerability to Cache Pollution:** Non-reused scans evict active blocks.
- **Thrashing Failure:** 0% hit rate in loops exceeding cache size ($N+1$).
- **Static Design:** Cannot adapt to dynamically shifting access workloads.

Objective

We aim to introduce intelligent adaptability to the classic Least Recently Used (LRU) policy.

Core Objective

Goal: Add a lightweight Reinforcement Learning (RL) layer on top of LRU.

- Learn the optimal eviction strategy per workload dynamically.
- Decide whether to evict the LRU candidate or give it a "second chance" (keeping it).

Implementation Architecture

We integrate the RL decision engine seamlessly into the cache eviction loop.

Core Components:

- **Cache Simulator:** Tracks state transitions and maintains associativity.
- **LRU Policy:** Provides the baseline metric and baseline fallback strategy.
- **Q-Learning Agent:** Evaluates candidate states and decides the optimal action.

LRU vs RL-LRU Comparison

The key difference lies in the flexibility of the eviction decision process.

Standard LRU

- Directly and blindly evicts the least recently used block.
- Rigid and inflexible; fails to adapt to patterns like cyclic or scan workloads.

RL-LRU

- LRU selects the candidate for eviction.
- RL agent decides the eviction action based on learned patterns (`EVICT` or `KEEP`).

Markov Decision Process (MDP) Basics

Before formulating our RL approach, we define the cache environment as a standard MDP.

State (S)

The current situation or context of the cache block environment.

Action (A)

The choices available to the agent (evict vs keep).

Reward (R)

Feedback signal indicating successful caching decisions.

Policy (π)

The strategy mapping states to the optimal eviction action.

The agent's goal is to learn a policy (π) that maximizes cumulative future rewards.

RL Formulation

We framed the cache eviction decision as a compact Markov Decision Process (MDP).

State Space

`(frequency_bucket, recent_hit)`

- Compact representation of the LRU candidate block.
- Prevents state-space explosion and speeds up training.

Action Space

- **EVICT:** Perform standard LRU eviction.
- **KEEP:** Second-chance protection (move candidate to MRU, evaluate next-oldest).

Q-Learning Implementation Details

Our RL agent utilizes Tabular Q-Learning to learn the optimal policy.

Q-Table Structure

A dictionary mapping the `(frequency, recent_hit)` state to expected rewards for `EVICT` and `KEEP`.

e-Greedy Strategy

Balances exploration (random actions) and exploitation (highest Q-value action).

Q-Value Update Rule

Uses the Temporal Difference (TD) Bellman equation to update expected values based on delayed rewards:

Allows iterative refinement of policy values as feedback is received from the environment.

The Reward Problem

Immediate feedback fails in caching environments due to delayed reuse patterns.

The Core Challenge: Eviction quality can only be determined by future access patterns, not immediate hits or misses.

Immediate reward loops can train the agent to make bad evictions that cause future misses.

Flawed Immediate Reward Loop

1. Agent evicts block **A**.
2. Next request is **B** (causes a hit/miss elsewhere).
3. Agent gets `+1` reward for immediate successful request.
4. Shortly after, block **A** is requested → **MISS!**

Proposed Solution: Ghost Cache

We delay rewards using a bounded history queue (Ghost Cache) to capture future reuse.

Mechanism: A queue storing the metadata (`Block` , `State` , `Action`) of evicted blocks.

Allows accurate feedback attribution based on whether the evicted block is requested again soon.

Reused Block → Penalty (-1)

Block requested while in ghost cache (a "near miss" due to early eviction).

Expired Block → Reward (+1)

Block fell out of ghost cache without being requested (good eviction choice).

Evaluation Workloads

We evaluated the system using five distinct workload dimensions to test robustness.

Each workload targets a specific caching behavior, evaluating stability, adaptability, and resilience to cache pollution.

Workload	Evaluation Purpose
Stable Working Set	Baseline hit rate stability
Cache+1 Loop	Severe LRU thrashing weakness
Hot Set + Scan	Pollution resistance (scans)
Zipfian	Highly skewed popularity distribution
Phase Shift	Dynamic adaptiveness over time

Evaluation Results (Cache Size: 4)

RL-LRU delivers dramatic gains in thrashing scenarios while remaining safe elsewhere.

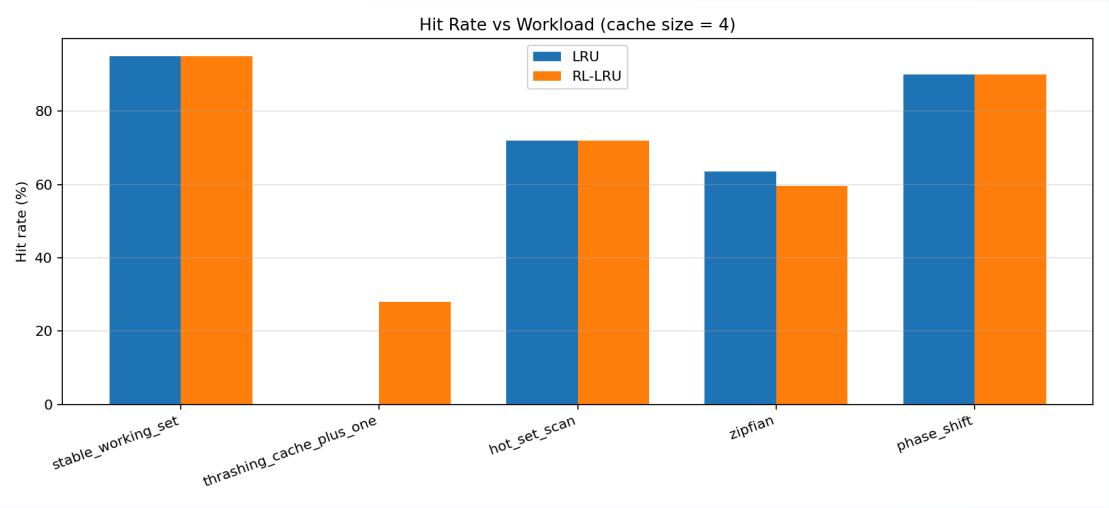
Workload	LRU	RL-LRU	Δ Hit Rate
Stable Working Set	95.0%	95.0%	0.0%
Cache+1 Loop	0.0%	26.0%	**+26.0%**
Hot Set + Scan	72.0%	72.0%	0.0%
Zipfian	63.5%	60.5%	**-3.0%**
Phase Shift	90.0%	90.0%	0.0%

Key Takeaways

- **Loop Bypassing:** Captures +26% gains under severe thrashing loops.
- **Zipfian Penalty:** Slightly underperforms (-3.0%) due to learning delays in high-entropy states.
- **Safe Guardrails:** Reverts to standard LRU performance in baseline workloads.

Hit Rate vs Workload

Comparison of hit rates across different workloads with a cache size of 4.

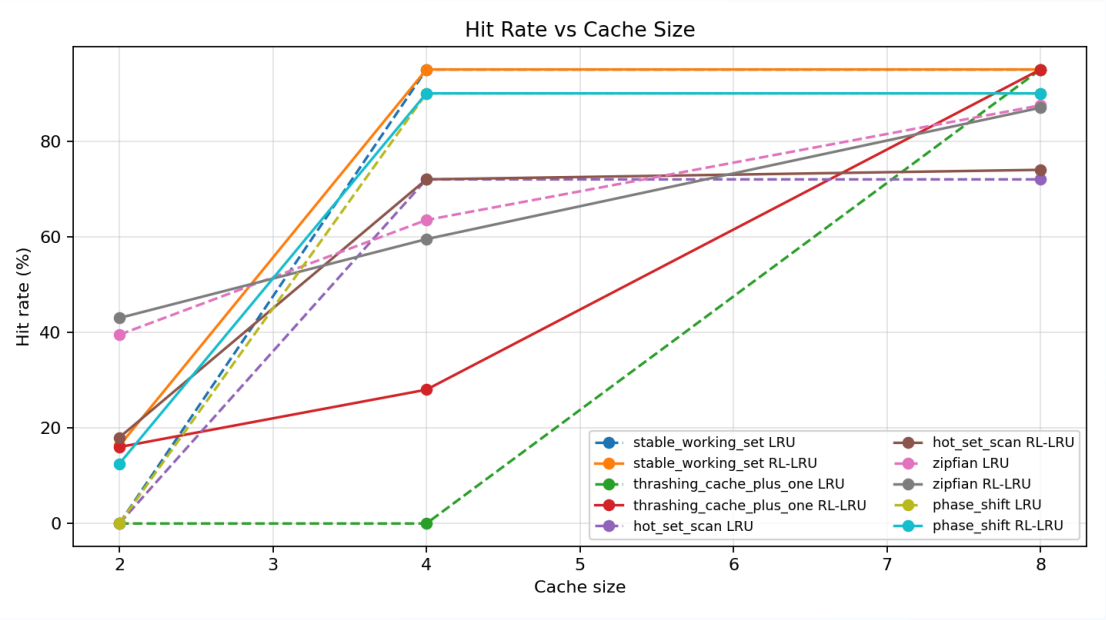


Key Insights

- **Thrashing Overcome:** RL-LRU successfully avoids complete thrashing (0% hit rate) in Cache+1 loops.
- **Robustness:** It retains full baseline performance in stable and shifting environments.
- **Safe Learning:** The agent learns to bypass eviction without causing collateral damage.

Hit Rate vs Cache Size

Hit rate progression as cache capacity scales from 2 to 8 blocks.

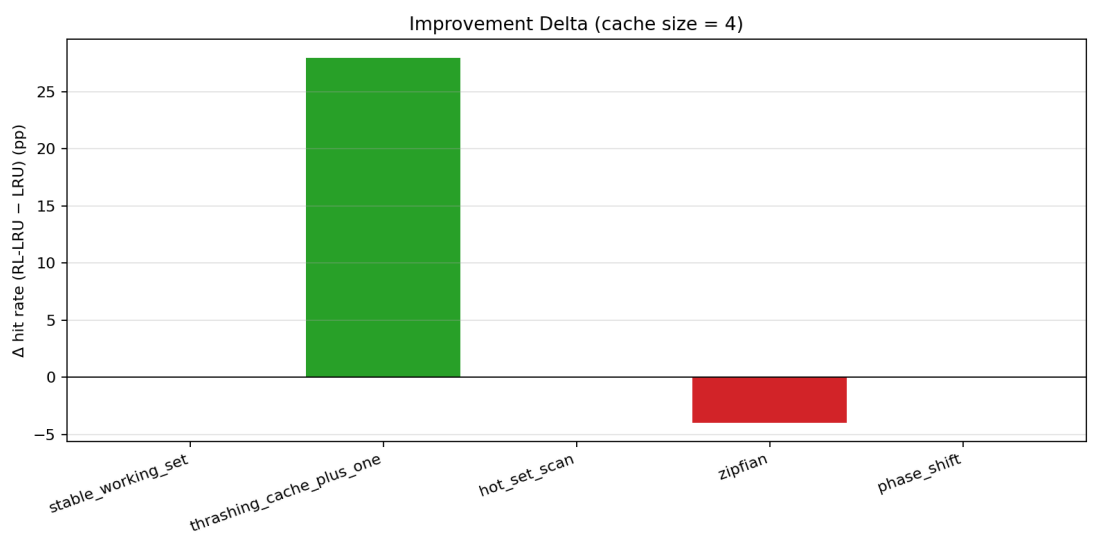


Key Insights

- **Critical Value:** The largest improvements occur at smaller cache sizes, where eviction choices are most crucial.
- **Natural Convergence:** As capacity increases, LRU naturally hits more frequently, narrowing the performance gap.

Delta Comparison

Performance delta between RL-LRU and standard LRU.



Key Insights

- **Positive Delta:** Confirms that the second-chance KEEP action successfully breaks strict LRU loops.
- **Zipfian Distribution:** Highlight the difficulty of learning from noisy delayed rewards in randomized access sequences.

Future Work

Our next steps to build on these findings:

1. **State Space Expansion:**
   Incorporate Reuse Distance approximations to capture deeper temporal patterns.
2. **Real Memory Traces:**
   Evaluate performance on real-world SPEC CPU and database access traces.
3. **Advanced Paradigms:**
   Explore Contextual Bandits and Deep Q-Networks (DQN) for complex states.